

Section 2: Byte-Pair Tokenizer

2.1 The Unicode Standard

(a)

```
>>> chr(0)
'\x00'
```

(b)

The printed representation returns an empty line, while the character's string representation returns

```
>>> chr(0)
'\x00'
```

(c)

`chr(0)` encodes a NULL byte, so nothing is printed in its position when it occurs in printed text.

2.2 Unicode Encodings

(a)

UTF-8 encoded bytes are shorter per character; UTF-32 can have as many as 8 bytes per Unicode character. The byte vocabulary in both UTF-32 and UTF-16 is also much larger than the manageable 256.

(b)

Passing

```
>>> aigue_encoded = "é".encode("utf-8")
```

to the incorrect function yields a `UnicodeDecode` error. This decode function assumes that one byte corresponds to a Unicode character, which is not always true.

(c)

The bytestring `b'\xe3\x81'` doesn't correspond to any Unicode character; it is a subset of `b'\xe3\x81\x93'` which corresponds to 'こ'

Merging algorithm

1. For each word, get word splits, for example the becomes `[b't', b'h', b'e']`. Store these in a word split dict with the word as the key and the array as the value. The counts of each word is already stored in the Counter object from before.
2. For each loop (to add a word to the vocabulary):
 - reinitialize pair counter
 - count pairs across all current word splits
 - pick best pair by lexicographic order
 - record merge and add to vocabulary
 - apply merge everywhere (for each word, recompute pairs. Look at whether any pair exists in the merges recorded; if so, replace the pair in the word splits with a merged version)

2.7 Tokenizer Experiments

(a)

Tiny Stories Compression Ratio: 4.04 bytes/token

OpenWebText Compression Ratio: 4.51 bytes/token

(b)

(Wrongly Encoded) Tiny Stories Compression Ratio: 3.90 bytes/token

(Wrongly Encoded) OpenWebText Compression Ratio: 3.41 bytes/token

Compression ratio goes down.

(c)

Average TinyStories tokenizer throughput: 1492647.54 bytes/second

Average OWT tokenizer throughput: 1361865.32 bytes/second

Taking the average throughput to be about 1.36 MB/s (obtained by averaging the two figures above), we obtain that it would take about 7 days 4 hours 24 minutes 17 seconds to tokenize the Pile dataset.

(d)

Serializing to `uint16` is advisable for two reasons: it's the smallest standard primitive that's expressive enough to handle a vocab size of up to $2^{16} = 65536$. Also, we use an unsigned format because all our indices are non-negative.

Section 3: Transformer Language Architecture

Transformer LM Resource Accounting

(a)

Trainable params: 1,640,452,800

Memory consumed by GPT-2 XL: 6.11 GB

(b)

RMS Norm: No matrix multiplication FLOPs, since we're doing elementwise operations/ reductions.

RoPE:

- For any vector that we apply RoPE to, we have $B \cdot S \cdot \frac{D}{2}$ pairs across all heads which we rotate, that is, $B \cdot S \cdot \frac{d_k}{2}$ for each head.
- Each rotation of pairs is a 2×2 matrix applied to a 2×1 vector, which yields 6 FLOPs per rotation.
- So we have $6 \cdot (B \cdot S \cdot \frac{D}{2}) = 3 \cdot B \cdot S \cdot D$
- Recall that we apply RoPE to both query and key vectors, so total FLOPs in RoPE is $6 \cdot B \cdot S \cdot D$
- We have one RoPE matrix for all layers

Causal Multihead Attention: We have:

- Each input projection matrix (Q, K, V) is responsible for $2BSD^2$ FLOPs.
- Scaled Dot Product Attention is responsible for $4BS^2D$ (matmul) FLOPs, since multiplying keys by queries to get weights takes $2BS^2D$ FLOPs, and multiplying the resulting weights by values takes another $2BS^2D$
- Output projection matrix is responsible for $2BSD^2$ FLOPs
- So total is $\text{num_layers} \cdot (8BSD^2 + 4BS^2D)$ FLOPs (note that $D = D_{\text{model}}$)

Feed-Forward Network : We have three matrices, two of which take in data of size d_{model} and output matrices of dimension d_{ff} , and the output layer which takes in the matrix from the hidden layer of dimension d_{ff} and projects it back to dimension d_{model} .

- The first layer accounts for $4BSD_{\text{model}}D_{\text{ff}}$ FLOPs, where each matrix does $2BSD_{\text{model}}D_{\text{ff}}$ FLOPs
- The second layer accounts for $2BSD_{\text{ff}}D_{\text{model}}$

- So total from all attention layers is $\text{num_layers} \cdot 6BS D_{\text{model}} D_{\text{ff}}$

Language Model Head: This accounts for $2 \cdot B \cdot S \cdot D_{\text{model}} \cdot V$ where V is vocab size.

Plugging in $S = 1024$, $D = D_{\text{model}} = 1600$, $D_{\text{ff}} = 4288$, $B = 1$, we get the following counts:

- **RoPE:** $\approx 9.8 \times 10^6$ FLOPs
- **Attention:** $\approx 2.77 \times 10^{10}$ FLOPs
- **FFN:** $\approx 4.2 \times 10^{10}$ FLOPs

which yields a total of

$\approx$

(c)

FFN requires the most FLOPs, with attention following close behind.

(d)

GPT-2 XL FLOPs: 3.517e12

GPT-2 Large FLOPs: 1.769e12

GPT-2 Medium FLOPs: 8.302e11

GPT-2 Small FLOPs: 2.917e11

Proportions for GPT-small

- attn – 9.664e10 FLOPs – 33.13%
- ffn – 1.160e11 FLOPs – 39.76 %
- rope – 4.719e06 FLOPs – 0.00 %
- head – 7.905e10 FLOPs – 27.10 %

Proportions for GPT-medium

- attn – 3.092e11 FLOPs – 37.25 %
- ffn – 4.155e11 FLOPs – 50.05 %
- rope – 6.291e06 FLOPs – 0.00 %
- head – 1.054e11 FLOPs – 12.70 %

Proportions for GPT-large

- attn – 6.765e11 FLOPs – 38.25 %
- ffn – 9.603e11 FLOPs – 54.30 %
- rope – 7.864e06 FLOPs – 0.00 %
- head – 1.317e11 FLOPs – 7.45 %

Observation: as model size gets larger, a larger percentage of the total FLOPs are accounted for in the attention and Feed-Forward layers. RoPE always accounts for a negligible amount of FLOPs.

(e)

GPT XL with context length 1024: 3.517e12. Proportions:

- attn – 1.329e12 FLOPs – 37.78 %
- ffn – 2.023e12 FLOPs – 57.53 %
- rope – 9.830e06 FLOPs – 0.00 %
- head – 1.647e11 FLOPs – 4.68 %

GPT XL with context length 16834: 1.336e14 FLOPs. Proportions:

- attn – 9.857e13 FLOPs – 73.79 %
- ffn – 3.237e13 FLOPs – 24.24 %
- rope – 1.573e08 FLOPs – 0.00 %

- head – 2.635e12 FLOPs – 1.97 %

Increasing context length also increases FLOPs (a context length increase of $\approx 16.0x \Rightarrow$ a 37x increase in FLOPs). It also means that a higher percentage of the FLOPs occur in the attention layer.